

ANSIBLE HANDLERS :

- Ansible modules are designed to be idempotent. This means that in a properly written playbook, the playbook and its tasks can be run multiple times without changing the managed host unless they need to make a change to get the managed host to the desired state.
- However, sometimes when a task does make a change to the system, a further task may need to be run. For example, a change to a service's configuration file may then require that the service be reloaded so that the changed configuration takes effect.
- Handlers are tasks that respond to a notification triggered by other tasks.
- Tasks only notify their handlers when the task changes something on a managed host.
- Each handler has a globally unique name and is triggered at the end of a block of tasks in a playbook.
- If no task notifies the handler by name, then the handler will not run.
- If one or more tasks notify the handler, the handler will run exactly once after all other tasks in the play have completed.
- Because handlers are tasks, administrators can use the same modules in handlers that they would use for any other task.
- Normally, handlers are used to reboot hosts and restart services.
- **Execution timing:** They run at the end of a play (after all tasks finish), ensuring efficiency and idempotency.
- **Purpose:** Prevent unnecessary restarts or reloads by acting only when something changes.
- **Common uses:** Restarting services (e.g., Apache, Nginx), reloading firewall rules, refreshing configuration files.

Some important points about Handlers

- 1: Handlers always run in the order specified by the handlers section of the play. They do not run in the order in which they are listed by notify statements in a task, or in the order in which tasks notify them.
- 2: Handlers normally run after all other tasks in the play complete. A handler called by a task in the tasks part of the playbook will not run until all tasks under tasks have been processed. (There are some minor exceptions to this.)

- 3: Handler names exist in a per-play namespace. If two handlers are incorrectly given the same name, only one will run.
- Even if more than one task notifies a handler, the handler only runs once. If no tasks notify it, a handler will not run.
- 4: If a task that includes a notify statement does not report a **changed** result (for example, a package is already installed and the task reports ok), the handler is not notified. The handler is skipped unless another task notifies it. Ansible notifies handlers only if the task reports the **changed** status.
- 5: Ansible treats the notify statement as an array and iterates over the handler names.

Example:

1: Install and enable **nginx & FirewallD**

2: Create a dir **"/websites"** on managed host.

3: Copy custom configuration file from Control Node **files/default.conf** to **/etc/nginx/conf.d/default.conf**

4: Copy **src:** files/index.html **dest:** /website/index.html.

5: Allow http & https in firewall.

6: After any change for both nginx and firewallD , use handlers to restart/reload services.

➔ files/default.conf looks like:

```
```bash
server {
 listen 80;
 server_name localhost;

 location / {
 root /websites;
 index index.html index.html;
 }
}
```

## →PlayBook:

``yaml

- name: Install and configure nginx with custom website

hosts: centnodes

become: yes

vars:

web\_service: nginx

config\_src: files/default.conf

config\_dest: /etc/nginx/conf.d/default.conf

website\_dir: /websites

tasks:

# 1. Install and enable nginx & firewall

- name: Install required packages

package:

name:

- "{{ web\_service }}"

- firewall

state: present

- name: Enable and start nginx

service:

name: "{{ web\_service }}"

state: started

enabled: yes

- name: Enable and start firewalld

service:

name: firewalld

state: started

enabled: yes

# 2. Create /websites directory

- name: Create website directory

file:

path: "{{ website\_dir }}"

state: directory

owner: "{{ web\_service }}"

group: "{{ web\_service }}"

mode: '0755'

# 3. Copy custom nginx configuration

- name: Copy custom nginx configuration

copy:

src: "{{ config\_src }}"

dest: "{{ config\_dest }}"

owner: root

group: root

mode: '0644'

notify: restart nginx

# 4. Copy index.html to /websites

- name: Copy custom index page

copy:

src: files/index.html

dest: "{{ website\_dir }}/index.html"

owner: "{{ web\_service }}"

group: "{{ web\_service }}"

mode: '0644'

# 5. Allow HTTP & HTTPS in firewall

- name: Allow HTTP and HTTPS services

firewalld:

service: "{{ item }}"

permanent: yes

state: enabled

loop:

- http

- https

notify: reload firewalld

# 6. Handlers for restart/reload

handlers:

- name: restart nginx

service:

name: "{{ web\_service }}"

state: restarted

- name: reload firewalld

firewalld:

state: reloaded

'''